T2112
Développement informatique III (Pratique)

Déploiement
TP 05

Auteur(s) :
Jonathan Noël

Dernière mise à jour le
12 mars 2026

Table des matières

1 Introduction **2**

2 Préparation du serveur distant **2**
2.1 Connexion au serveur 2
2.2 Installation des prérequis 2

3 Préparation du backend de production **2**
3.1 Base de données Supabase - Postgres 3
3.2 Migrations Sequelize 3

4 Déploiement du Backend - via Docker **5**
4.1 Création du Dockerfile 5
4.2 Build & Push manuel vers DockerHub 5
4.3 Lancement sur le VPS 5
4.4 [Optionnel] Vérification du Firewall VPS 6

5 Déploiement du Frontend React via SCP **6**
5.1 Variables d'environnements dev - prod 6
5.2 Build de l'application 6
5.3 Transfert via SCP 7

6 Partie 4 : Introduction au CI/CD via GitHub Actions **7**
6.1 CI - Lancement des tests 7
6.2 CD - Déploiement du frontend 7
6.2.1 Configuration des Secrets GitHub 8
6.2.2 Création du Workflow de Déploiement 8

Légendes

👨‍🏫 Conseil du professeur.
ℹ️ Une information ...
🚨 Pré-requis pour le TP.
🖥️ Exercice à réaliser.
👓 Exercice d'observation ou de lecture.

1 Introduction

Jusqu'à présent, votre application (Frontend et Backend) tournait exclusivement sur votre machine locale (`localhost`). Le processus de déploiement consiste à mettre à disposition cette application sur un serveur distant afin qu'elle soit accessible aux utilisateurs finaux.

👩‍🏫 Vous utilisez certainement `concurrently` pour le lancement de vos 2 applications en développement, cette manière de faire n'est utilisable que dans ce cas-là.

Dans ce TP, nous allons déployer notre application "Monorepo" sur le VPS qui vous a été fourni dans le cadre du cours "Administration système et réseaux II". Nous utiliserons une stratégie mixte :

- **Le Backend :** Sera conteneurisé avec Docker pour garantir la portabilité et l'isolation.
- **Le Frontend :** Sera compilé localement puis transféré via la commande `scp` vers un serveur web (Nginx) sur le VPS.

Enfin, nous ferons nos premiers pas en DevOps en automatisant l'exécution de nos tests (CI) et du déploiement du frontend (CD) à l'aide de GitHub Actions.

ℹ️ Aidez-vous un maximum des documentations officielles liées aux technologies que vous utiliserez dans le tp. Si dans les 30 dernières minutes du TP vous n'arrivez pas à avancer, aidez-vous de l'IAg, mais pas avant !

2 Préparation du serveur distant

Vous avez reçu les accès (IP, utilisateur, mot de passe/clé SSH) pour votre VPS dans le cadre du cours d'admin. Vous avez certainement déjà travaillé sur le VPS et installé Docker, mais assurez-vous quand même que les services suivants soient installés.

2.1 Connexion au serveur

Ouvrez un terminal et connectez-vous à votre serveur via SSH, pour rappel :

```
ssh utilisateur@adresse_ip_du_vps
```

👩‍🏫 Évidemment, c'est préférable d'ajouter votre clé publique dans les `authorized_keys` sur le VPS. Si pas, il est conseillé de vous connecter par mot de passe et ensuite ajoutez une clé

2.2 Installation des prérequis

Une fois connecté, mettez à jour le système et installez les outils nécessaires (Docker pour l'API, Nginx pour servir le Frontend) :

```
sudo apt update && sudo apt upgrade -y
sudo apt install docker.io nginx -y
```

et vérifiez que les services sont actifs :

```
sudo systemctl status docker
sudo systemctl status nginx
```

3 Préparation du backend de production

Jusqu'à présent, votre API utilisait une base de données locale SQLite. Une fois sur le VPS, votre conteneur Docker aura besoin d'une base de données accessible. Plutôt que d'installer et de maintenir un serveur de base de données sur le VPS, nous allons utiliser une solution "Database-as-a-Service".

3.1 Base de données Supabase - Postgres

Nous utiliserons **Supabase**, une plateforme qui fournit des bases de données PostgreSQL gratuites, idéales pour les petits projets pédagogiques.

1. Rendez-vous sur `https://supabase.com` et créez un compte (avec GitHub).
2. Créez un nouveau projet (choisissez la région la plus proche, ex : Frankfurt ou Paris) et définissez un mot de passe sécurisé pour la base de données.
3. Une fois le projet prêt, allez dans le **Project Overview**.
4. Sous le nom de la DB, il y a une chaine de connexion, passez la souris dessus et vous aurez un **Direct Connection String**.
5. Copiez la chaîne de connexion. Elle devrait ressembler à ceci :

```
postgresql://postgres.votre_projet:[MOT_DE_PASSE]@aws-0-eu-central-1.
    pooler.supabase.com:6543/postgres
```

Cette URL sera utilisée comme variable d'environnement (`DATABASE_URL`) lors du lancement de votre conteneur Docker.

👩‍🏫 N'oubliez pas de remplacer `[MOT_DE_PASSE]` par le mot de passe défini à l'étape 2

3.2 Migrations Sequelize

ℹ️ Comme précisé pendant le cours théorique, il est indispensable de penser aux migrations https://sequelize.org/docs/v6/other-topics/migrations/ de la base de données une fois que celle-ci passe en production. En effet, les modifications apportées à la structure de la base de données ne doit pas être effectuée en écrasant la DB existante, mais en la faisant évoluer grâce aux migrations. Ces migrations sont dans un ensemble de fichiers .js qui reprennent les évolutions de la DB a effectuer.

FIGURE 1 – Exemple d'exécution de migrations, une par une et dans **l'ordre de timestamp**.

Votre base de données Supabase est actuellement vide. Avant de lancer l'API sur le VPS, vous devez y créer vos tables. Commencez par envoyer vos migrations sur le serveur Supabase. Depuis votre machine locale, dans le dossier `server/`, il faut dans un premier temps initialiser la configuration sequelize

```
npx sequelize-cli init
# Il faut aussi installer le paquet postgres
npm i pg
```

Il est aussi important d'aller mettre à jour le service utilisé par votre connecteur de db sans le fichier database.ts. Remplacez sqlite par postgres.

```
const sequelize = process.env.DATABASE_URL
  ? new Sequelize(process.env.DATABASE_URL, {
      dialect: "postgres",
      dialectOptions: {
        ssl: {
          require: true,
          rejectUnauthorized: false,
        },
      },
      logging: false,
    })
  : new Sequelize({
      username: process.env.DB_USER || "postgres",
      password: process.env.DB_PASSWORD,
      database: process.env.DB_NAME || "postgres",
      host: process.env.DB_HOST,
      port: Number(process.env.DB_PORT || 5432),
      dialect: "postgres",
      dialectOptions: {
        ssl: {
          require: true,
          rejectUnauthorized: false,
        },
      },
      logging: false,
    });
```

Ensuite, adaptez toutes les parties du fichier de configuration sequelize (config/config.json) avec vos informations, puis exécutez vos migrations.

```
  "development": {
    "username": "postgres",
    "password": "unMagnifiquePassword",
    "database": "postgres",
    "host": "db.xxxxxx.supabase.co",
    "port": 5432,
    "dialect": "postgres",
    "dialectOptions": {
      "ssl": {
        "require": true,
        "rejectUnauthorized": false
      }
    }
  },
```

```
npx sequelize-cli db:migrate
```

🤦 Si vous avez l'erreur *"ERROR : module is not defined"*, vérifiez bien que vos fichiers de migrations soient sous format suivant :

```
'use strict';

/** @type {import('sequelize-cli').Migration} */
export default {
...
```

Vérifiez depuis le tableau de bord Supabase (Table Editor) que vos tables ont bien été créées.

4 Déploiement du Backend - via Docker

Le Backend contient votre logique métier et doit s'exécuter dans un environnement Node.js. Pour éviter le syndrome du "ça marche sur ma machine", nous allons utiliser Docker.

4.1 Création du Dockerfile

Dans votre dossier `server/`, créez un fichier nommé `Dockerfile` (sans extension) :

```
FROM node:18-alpine
WORKDIR /usr/src/app
COPY package*.json ./
RUN npm install
COPY . .
# On compile le TypeScript en JavaScript
RUN npm run build
ENV PORT=3000    # Très important !
EXPOSE 3000      # Très important !
CMD ["npm", "start"]
```

👨‍🏫 L'exposition est pour le conteneur, on liera le port du VPS par la suite, au run du conteneur.

4.2 Build & Push manuel vers DockerHub

1. Commencez par créer un compte sur DockerHub [1] si vous n'en avez pas encore.
2. Créez un répertoire sur le DockerHub
3. Construisez l'image locale :

   ```
   docker build -t votre_pseudo_dockerhub/mon-api:latest .
   ```

   👨‍🏫 Pour rappel, attention que le . à la fin est indispensable !

4. Connectez-vous à DockerHub dans votre terminal puis poussez l'image :

   ```
   docker login
   docker push votre_pseudo_dockerhub/mon-api:latest
   ```

4.3 Lancement sur le VPS

1. Retournez sur le terminal de votre VPS.
2. Récupérez l'image depuis DockerHub :

   👨‍🏫 Le `sudo` n'est indispensable que si vous n'avez pas ajouté votre utilisateur courant au groupe `docker`.

   ```
   sudo docker pull votre_pseudo_dockerhub/mon-api:latest
   ```

3. Lancez le conteneur.

   👨‍🏫 Attention aux variables d'environnement ! Ne mettez JAMAIS de données sensibles (clés secrètes, mots de passe de base de données) dans le code source sur GitHub. Passez-les au lancement du conteneur pour commencer. Dans ce cas, listez l'ensemble des variables dont vous avez besoin, ci-dessous n'est qu'un exemple.

   ```
   sudo docker run -d -p 3000:3000 -e NODE_ENV=production \\
   -e DB_PASS=monMotDePasse votre_pseudo_dockerhub/mon-api:latest
   ```

Vérifiez que votre API répond en allant sur http://adresse_ip_du_vps:3000/api/users

1. https://hub.docker.com/

4.4 [Optionnel] Vérification du Firewall VPS

🚨 Si vous n'avez pas encore configuré de firewall (ufw) pendant votre cours d'Admin, veillez à bien ouvrir aussi le port 22 quand vous activez le firewall, sinon votre connexion SSH sera bloquée ! 😱

Il se peut que la communication vers votre port 3000 ne fonctionne pas. Pour ce faire, il faut ajouter dans la configuration du firewall du VPS une règle qui définit que les communications sur le port 3000, tcp et udp de préférence, soient possible.

ℹ️ Quand vous aurez tout configuré au cours d'administration, on passera tout sous domaine.

5 Déploiement du Frontend React via SCP

Contrairement au Backend qui doit s'exécuter en continu, le Frontend React n'est qu'un ensemble de fichiers statiques (HTML, CSS, JS) une fois compilé.

5.1 Variables d'environnements dev - prod

Actuellement, dans votre code React, vous avez probablement codé l'URL de votre API "en dur" (ex : `fetch('http://localhost:3000/api/...')`). C'est une mauvaise pratique : en production, l'API ne sera plus sur `localhost` mais sur l'IP de votre VPS.

Vite gère très facilement les environnements grâce aux fichiers `.env`.

1. Dans le dossier `client/`, créez deux fichiers distincts :

   Fichier `.env.development` :

   ```
   VITE_API_URL=http://localhost:3000/api
   ```

   Fichier `.env.production` :

   ```
   VITE_API_URL=http://VOTRE_IP_VPS:3000/api
   ```

2. Dans vos composants React (ex : `App.tsx`), si vous aviez déjà des appels fetch (ce qui devrait être le cas) remplacez l'URL codée en dur par la variable d'environnement fournie par Vite :

   ```
   // Avant :
   fetch('http://localhost:3000/api/users')

   // Apres :
   fetch(`${import.meta.env.VITE_API_URL}/users`)
   ```

Lorsque vous lancez `npm run dev`, Vite chargera le fichier `development`. Lorsque vous lancerez `npm run build` (à l'étape suivante), Vite remplacera automatiquement la variable par l'IP de votre VPS présente dans le fichier `production`.

5.2 Build de l'application

Sur votre machine locale, allez dans le dossier `client/` :

1. Assurez-vous que l'URL de votre API pointe désormais vers l'IP de votre VPS (ex : `http://adresse_ip_du_vps:3000/api`) et non plus `localhost`.
2. Lancez le build :

   ```
   npm run build
   ```

Cela va générer un dossier `dist/` contenant les fichiers optimisés. C'est ce dossier qui contient le projet qui sera déployé sur le serveur distant.

5.3 Transfert via SCP

Nous allons envoyer ce dossier directement dans le répertoire que Nginx utilise par défaut pour servir les sites web (`/var/www/html`). Si vous êtes déjà à l'étape de création de virtualhost ou de gestion de plusieurs projets sous nginx, n'hésitez pas à exploiter cette force afin de ne pas écraser le projet sur `/var/www/html` qui est lié au cours d'admin.

Depuis le dossier racine de votre projet en local, lancez la commande suivante :

```
scp -r ./client/dist/* utilisateur@adresse_ip_du_vps:/var/www/html/
```

👨‍💻 Si vous avez une erreur de permission, transférez d'abord vers le dossier `/home/utilisateur/` puis déplacez les fichiers avec `sudo mv` depuis la console du VPS.

Votre interface React est maintenant accessible en tapant directement l'IP (ou le domaine si vous avez un domaine `ephec-ti` bien configuré) de votre VPS dans votre navigateur !

6 Partie 4 : Introduction au CI/CD via GitHub Actions

Maintenant que vous savez déployer manuellement, automatisons une partie du cycle de vie DevOps : les tests et le déploiement du frontend. L'objectif est de vérifier automatiquement que votre code fonctionne à chaque fois que vous poussez sur GitHub. Dans un second temps, si les tests passent bien, on peut déployer le code en production.

6.1 CI - Lancement des tests

1. À la racine de votre projet local, créez l'arborescence suivante : `.github/workflows/`.
2. Créez-y un fichier `ci.yml` :

```yaml
name: Continuous Integration - Tests

on: [push, pull_request] # Se declenche a chaque push

jobs:
  test-backend:
    runs-on: ubuntu-latest # Runner
    steps:
      - uses: actions/checkout@v3 # Actions
      - name: Set up Node.js
        uses: actions/setup-node@v3
        with:
          node-version: '18.x'
      - name: Install dependencies
        run: npm install
        working-directory: ./server
      - name: Run unit tests
        run: npm test
        working-directory: ./server # Execute les tests unitaires
```

3. Commitez et poussez ce fichier sur GitHub.
4. Allez dans l'onglet **"Actions"** de votre dépôt GitHub pour voir votre workflow s'exécuter automatiquement !
5. S'il y a une erreur d'exécution, prenez le temps de comprendre pourquoi et de débogger jusqu'à ce que tout fonctionne correctement.

6.2 CD - Déploiement du frontend

Faire un `scp` manuel à chaque fois que vous modifiez le design de votre application est fastidieux. Le but du **Déploiement Continu (CD)** est d'automatiser cette tâche. Nous allons configurer GitHub Actions pour qu'il compile et envoie automatiquement la nouvelle version du Frontend sur le VPS à chaque fois qu'un `push` est fait sur la branche `main`.

6.2.1 Configuration des Secrets GitHub

Pour que GitHub puisse se connecter à votre VPS sans exposer vos mots de passe publiquement, nous devons utiliser les **Secrets**.

1. Allez sur la page de votre dépôt sur GitHub.
2. Cliquez sur **Settings** > **Secrets and variables** > **Actions**.
3. Créez les *New repository secrets* suivants :
   - `SERVER_IP` : L'adresse IP de votre VPS.
   - `SERVER_USER` : Votre nom d'utilisateur sur le VPS.
   - `SSH_PRIVATE_KEY` : Le contenu de votre clé SSH privée (qui vous permet de vous connecter au VPS).

6.2.2 Création du Workflow de Déploiement

Dans le dossier `.github/workflows/`, créez un nouveau fichier nommé `cd-frontend.yml` :

```yaml
name: Continuous Deployment - Frontend

on:
  push:
    branches:
      - main # Ne se declenche que sur la branche principale

jobs:
  deploy-frontend:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3

      - name: Setup Node.js
        uses: actions/setup-node@v3
        with:
          node-version: '18.x'

      - name: Install dependencies and Build
        run: |
          npm install
          npm run build
        working-directory: ./client

      - name: Deploy to VPS via SCP
        uses: appleboy/scp-action@v0.1.4
        with:
          host: ${{ secrets.SERVER_IP }}
          username: ${{ secrets.SERVER_USER }}
          key: ${{ secrets.SSH_PRIVATE_KEY }}
          source: "client/dist/*"
          target: "/var/www/html/"
          strip_components: 2
```

👨‍🏫 Le paramètre `strip_components : 2` est très pratique ici. Sans ça, l'action SCP créerait littéralement l'arborescence `/var/www/html/client/dist/...` sur le VPS. En mettant 2, on lui dit d'ignorer les 2 premiers dossiers du chemin source pour ne copier que le contenu final

Testez votre pipeline : Modifiez un texte visible dans votre application React (ex : un titre `<h1>`), faites un `commit` et poussez sur la branche `main`. Allez dans l'onglet **Actions** sur GitHub, attendez que le job se termine, puis rafraîchissez la page de votre VPS dans le navigateur. Le texte a dû se mettre à jour tout seul !

S'il y a une erreur d'exécution, prenez le temps de comprendre pourquoi et de débogger jusqu'à ce que tout fonctionne correctement.